

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

The minimum cost of reaching (m, n) from $(0, 0)$ is

$$\frac{m(m+1)}{2} + n.$$

The cost only depends on the x being left, so take all n north steps while still at $x = 0$, where each costs 1 and they total n , then the m east steps leave from $x = 0, 1, \dots, m-1$ in turn and cost $1, 2, \dots, m$; writing that series forwards and backwards pairs it into m columns of $m+1$, so it sums to $m(m+1)/2$. One path achieving the minimum is to take all n steps north from the origin first and then all m steps east, and it is the only one, since any north step taken after moving east costs more than 1 because it leaves from $x > 0$, and a detour into negative x adds at least one step west and one step back east, each costing at least 1, without making any other step cheaper.

1.b

1. False. The states go on forever, but UCS takes them in cost order and every step costs at least 1, so anything cheaper than the goal is only so many steps out from the origin. There are only so many of those, so it gets to the goal and stops.
2. False. It does give back the minimum cost path, but it looks outside the box on the way. $(-1, 0)$ and $(0, -1)$ both cost 1 to get to, which is cheaper than most goals, so they get processed even though they sit outside $0 \leq x \leq m$, $0 \leq y \leq n$.
3. False. It returns the minimum cost path, but it can also look at places costing exactly the same as the minimum rather than less. When things tie, the queue decides who goes first. For the goal $(1, 0)$, which costs 1, the states $(-1, 0)$, $(0, -1)$ and $(0, 1)$ also cost 1 and come out before it.

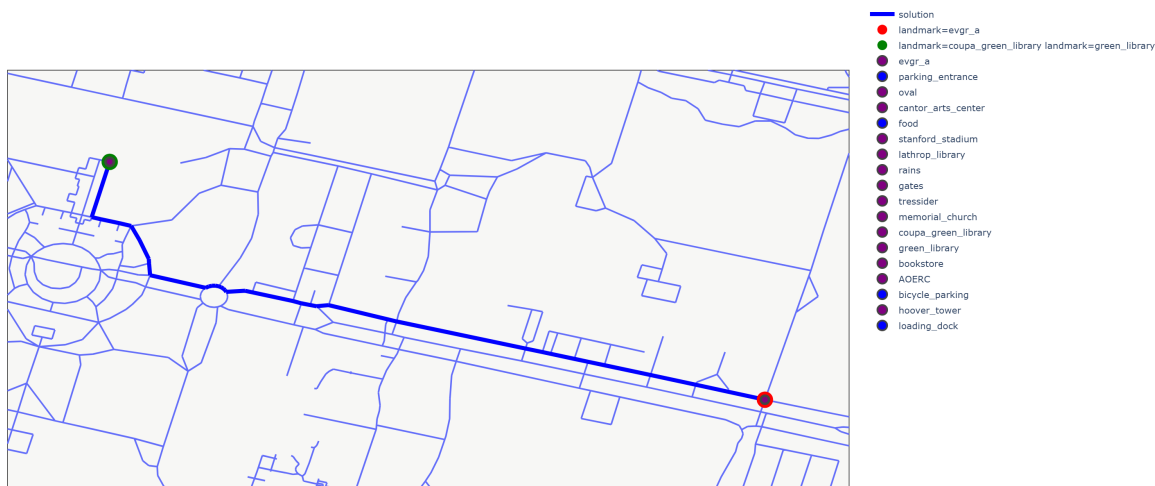
1.c

1. True. Adding a connection only gives you more paths, it never takes one away, so the minimum can drop or stay where it is but it cannot go up.
2. False. An action coming out of a state the start cannot get to is not on any path from start to goal, however cheap it is. Say $A \xrightarrow{5} C$ is the only way from start A to goal C , and $D \xrightarrow{-1000} E$ sits off on its own, then that cheap action never shows up. Same if it leads into a dead end.
3. False. Adding 1 to every action charges a path by how many edges it has, so the longer ones get hit harder and the order can flip. Route 1 is $A \xrightarrow{10} Z$ with one edge, route 2 is $A \xrightarrow{2} B \xrightarrow{2} C \xrightarrow{2} D \xrightarrow{2} Z$ with four. Before, route 1 costs 10 and route 2 costs 8, so route 2 wins. After, route 1 costs 11 and route 2 costs 12, so route 1 wins.

2.b

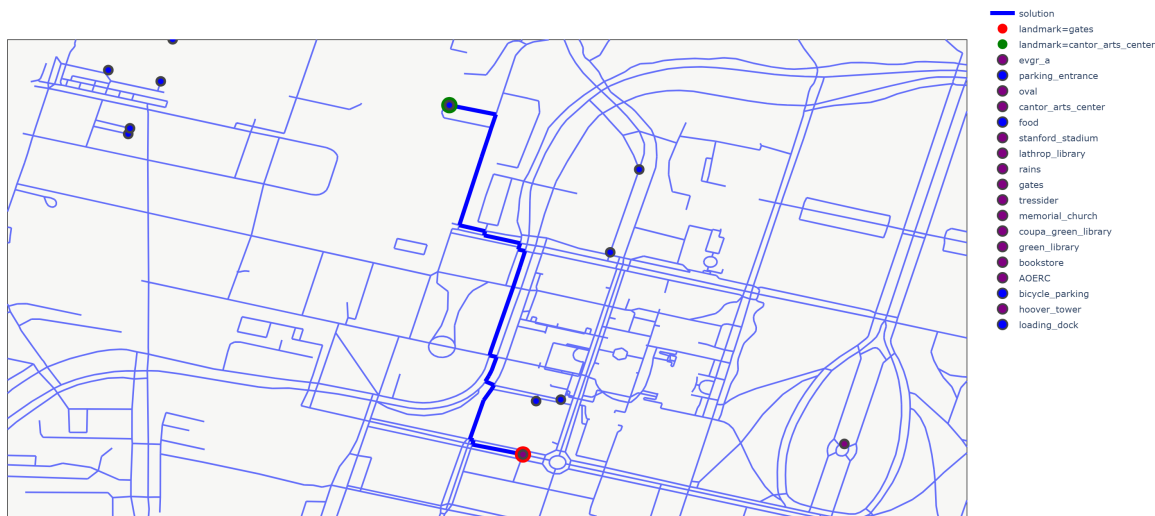
The first route start at EVGR-A with end tag `landmark=green_library`, which comes out around 606 m over 38 locations.

2b (useful): EVGR-A to Green Library



The second route start at Gates with end tag `amenity=food`, about 659 m, and it finishes at the Cantor Arts Center.

2b (undesirable): Gates to the nearest amenity=food



What I got out of it is that the cost is just distance, so anything that actually slows a walk down is free as far as the search is concerned. My EVGR-A route is around 606 m and goes through a couple of marked crossings, while the Gates one is roughly the same length but goes through about ten of them plus a bus stop, so the second walk takes a fair bit longer in practice and the search has no way to tell them apart. The locations are also spaced pretty unevenly, anywhere from under 2 m to nearly 100 m apart on the same path, so the number of steps does not say much about how far it is.

What seems wrong is that asking Gates for the nearest `amenity=food` sends to the Cantor Arts Center, while Tressider, which is where we would actually go, is further out and never gets picked. There are only a handful of locations on the map with `amenity=food` at all, and a couple of them are the art museum and the football stadium, which happens because the tag is sitting on the landmark's own node, so one cafe somewhere inside a big building ends up labelling the whole thing as food. The assumption that breaks is that a tag on one node stands for the whole place, and that somewhere you can walk to is somewhere that is open.

2.c

If everyone plans with the same system, the users all get sent down the same shortest path, and the congestion that creates never shows up in the cost, because the cost is only distance. The people who do not use it are hit whenever their own route intersects that optimal path, so quiet side streets that were never built for through traffic end up carrying it, and they get none of the benefit since they never opted in. The mismatch is that the model prices an edge as a fixed number when the real price depends on how many people are on it at the time, and the system has no way of knowing that its own advice is what put them there. One way to reduce that is to put actual speed into the cost rather than distance alone, so a road that is slow or already saturated costs more and the system spreads people out instead of funnelling them all onto one route.

3.b

The maximum number of states UCS could visit is

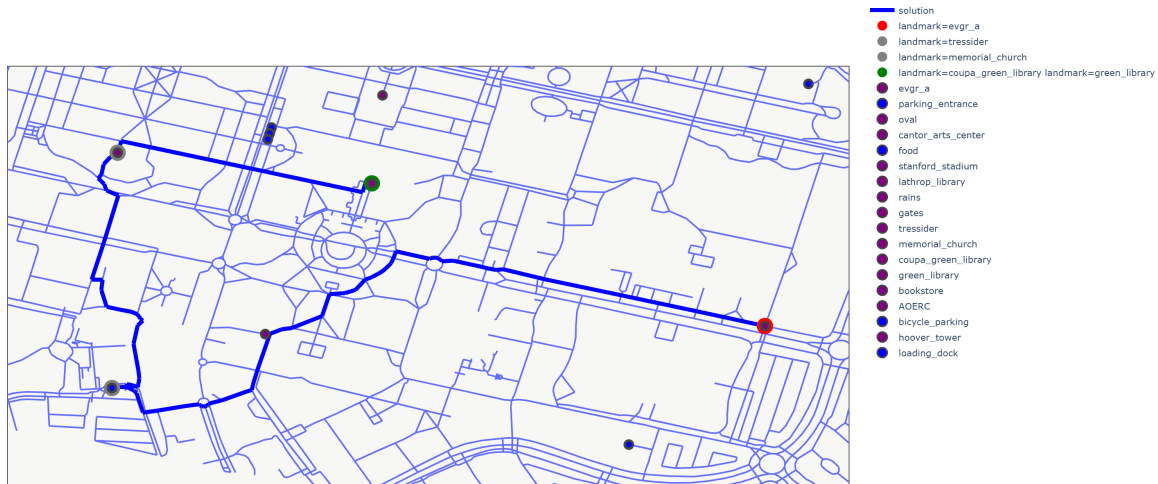
$$n \cdot 2^k.$$

A state is a location together with the set of waypoint tags covered so far, so there are n choices for the location, and for the memory each of the k tags is independently either covered or not, which gives 2^k possible sets. Multiplying the two independent choices gives $n \cdot 2^k$.

3.c

The waypoint tags are `landmark=memorial_church` and `landmark=tressider`, starting at EVGR-A with end tag `landmark=green_library`, and the path comes out at 1815.6 m over 130 locations.

3c: EVGR-A, via Memorial Church and Tressider, to Green Library



The order surprised me. I listed the church first, but the search went to Tressider at 991.5 m, then the church at 1464.8 m, then the library at 1815.6 m, which is the point of leaving the waypoints unordered: I do not have to guess the ordering, and here my guess would have been the more expensive one. What did match my expectation is the cost, since the same trip with no waypoints is 606.0 m, so adding two stops tripled it, which is what you get when the stops sit off the straight line between start and end.

What the path fails to capture is anything other than metres. It has no idea that Tressider closes in the evening, that the church is worth more time than a junction, or that I would rather eat before settling into the library than after, and it is one way, so it never accounts for getting home. A route that is optimal in distance can still be the wrong plan for the day.

3.d

Between a drop-off and the next pick-up I would insert a rest waypoint, tagged for a restroom, food, or somewhere to park for ten minutes, in the same way problem 4c joins a fake node to every end location at zero cost so the search finds whichever one is nearest. Here the cost would be small rather than zero, so a break is cheap enough that the route takes one but not so free that the driver is sent miles out of the way for it. To pick sensible waypoints I would want the driver's age and years of driving experience, since a new driver and a driver in their seventies do not need the same break pattern or the same length of shift. Those same two fields are exactly what a company could turn against them: knowing a driver's age lets dispatch quietly stop sending work to older drivers, which is age discrimination dressed up as routing, and it never has to be written down anywhere because it only shows up as fewer jobs.

5.a

Interactive tutoring session on Dynamic Programming:

<https://claude.ai/share/96fd17a6-37a0-4650-a6fc-3c82c68c815b>

5.b

Interactive tutoring session on Beam Search:

<https://claude.ai/share/c969fba9-4932-4670-9952-aa6afa912e3a>

6.a

My product is Claude Code, carried over from Assignment 1, and it is marketed through non-consumer channels rather than advertising, with new releases announced through Anthropic's own newsroom and social media and then carried outward by news outlets. The main surface is its product page at claude.com/product/claude-code, which leads with "Hand Claude a bug fix, test, or multi-day migration. Steer and review from your terminal, IDE, Slack, or web" and puts the one-line install straight on the page, so discovery and installation happen in the same step. Anthropic also runs its own developer conference, Code with Claude, which in 2026 ran in San Francisco, London and Tokyo with sessions from companies such as Spotify and Delivery Hero on how they restructured their engineering around it ([MIT Technology Review](#)). The other route in is the workplace rather than the individual: Claude Code is sold into businesses on Team and Enterprise plans at roughly \$20 to \$25 per seat plus usage, with admin controls, audit logs and SCIM (claude.com/pricing), so a lot of developers first meet it because their employer signed an agreement and it turned up in their existing workflow. Beyond that, most discovery is word of mouth inside developer communities, where Reddit threads trading multi-agent terminal setups spread the advanced features further than any official page does. Anthropic does advertise at brand level, including two Super Bowl spots in February 2026 attacking OpenAI's decision to put ads in ChatGPT, which lifted the Claude app from 41st to the top ten on the US App Store and daily active users by about 11 percent ([CNBC](#)), but that campaign was for Claude as a whole and never mentions Claude Code.

6.b

Anthropic states its mission on its company page as "Anthropic is an AI safety and research company. We build reliable, interpretable, and steerable AI systems" (anthropic.com/company). Its marketing runs through press releases and social media rather than consumer advertising, and the content of those releases lines up with the mission rather than cutting against it. The Life Sciences Verification Program, announced on 17 September 2026, gives verified researchers access to models with safeguards tuned for biology so they can do drug discovery and clinical work that the general models block (anthropic.com/news/life-sciences-verification-program), which is the steerable part of the mission shipped as a product rather than as a capability boast. Pushing the other way, the September 2026 threat intelligence report documents real misuse of Claude across seven categories including cyber espionage and election manipulation, published on the stated grounds that "we believe we have a responsibility to disclose malicious misuse of our services" (anthropic.com/threat-intelligence-report-september-2026). Publishing your own product's failures at that level of detail is the opposite of sensationalising it, and it is hard to square with a company that was overselling what its models can do. The one place the two rub is the Super Bowl campaign, where staying ad-free is a real safety position but is being used mainly as an attack on a competitor, so there the mission is doing marketing work as much as the marketing is serving the mission.

6.c

Support is written only. The help page states plainly that "We don't offer phone support" and that "Support is provided in writing through the support messenger and email", with an AI bot called Fin answering first and a human product support team reachable only on Pro, Max, Team and Enterprise, while free users get the documentation and the bot (support.claude.com). The process is asynchronous by design, so a specialist "investigates and replies in the same conversation" with response times varying by plan and severity, and there is no realtime phone or video route at all. For Claude Code specifically the community channels carry more of the load: the official documentation at code.claude.com is good on features, but the /bug command and the GitHub issue tracker at anthropics/claude-code, which currently holds over five thousand open issues, are where you find out whether a problem is yours or everyone's, alongside the Claude Developers Discord and the r/ClaudeAI subreddit.

6.d

Both queries went through the support messenger at support.claude.com.

Query 1, sent about Claude Code:

I use Claude Code on a Max plan and I'd like to know whether it can be pointed at a different model endpoint. I know Bedrock and Vertex are supported. Is there any supported way to use an OpenAI-compatible or OpenRouter endpoint, and if not, is that something on the roadmap? I'm asking because I want to compare outputs on the same repo without changing tools.

The reply:

Claude Code can connect through LLM gateways that expose a supported API format, but we don't support

routing Claude Code to non-Claude models through any gateway.

While you can configure Claude Code to use different Claude model endpoints (like Bedrock, Vertex, Microsoft Foundry, or through custom `ANTHROPIC_BASE_URL` settings), the gateway compatibility is specifically for accessing Claude models, not for switching to other model providers like OpenAI or OpenRouter.

For comparing outputs on the same repo, you'd need to use separate tools for non-Claude models. We don't have information about future roadmap plans for multi-model support in Claude Code.

It answered the actual question and drew a distinction I had not made myself, between reaching Claude models through a gateway and routing to another provider, and it said plainly that it had no roadmap information rather than giving me a vague maybe.

Query 2, sent about Claude itself:

I'm using Claude to learn the maths for a machine learning course, and I want it to tutor me rather than solve things for me. The problem I keep hitting is that when I'm deriving something, Claude already knows the final form and reasons backwards toward it, which looks like a derivation but doesn't help, because the answer is the one thing I don't have. Telling it to stop works for a few messages and then it drifts back. Where is the best place to put an instruction like that so it holds for a whole session, Project instructions, a custom style, or something else? And is there a recommended way to phrase it?

The reply:

For persistent tutoring instructions, Project instructions are your best option. They apply to all chats within that project, so you can keep your ML course work in one project with the tutoring behavior locked in.

Here's how to phrase it effectively. In your project instructions, try something like: "Act as a tutor, not a solver. When I'm deriving equations or working through problems, guide me with hints and questions rather than showing the final form or working backwards from the answer. Let me discover the solution step-by-step. If I get stuck, ask what I've tried or suggest the next small step, but don't reveal the destination."

You could also add this to your account-wide Instructions for Claude (Settings → Instructions for Claude) if you want this tutoring approach across all your conversations, not just in one project. Project instructions work on top of your account instructions, so you can combine both, general preferences account-wide and specific tutoring rules in your ML project.

This one came back with something I could act on rather than an acknowledgement, including a phrasing to paste straight in and where it sits relative to the account-wide settings. What it does not do is promise the behaviour will hold, which is fair enough, since the underlying thing is steering a model rather than flipping a switch.